DECA Lab Logbook — Spring Term

Lab 2 | Task 4: 16-bit Multiply in EEP1 Assembly

Task 4: 16-bit Multiply in EEP1 Assembly

Objectives

- Translate the C multiply algorithm into EEP1 assembly using if-then and while constructs
- Assemble and run the program on the EEP1 design to verify correct multiplication

Task 4. Using the method of Lecture 4 for if-then and while constructs, and the EEP1 instructions for addition, shifting, logical AND, translate the program in Figure 1 into assembly language, using registers R0, R1, R2, R3 for the 4 required variables. Write your program in a mul1.txt file and use eepassembler to turn this into a mul1.ram file containing machine code. Run the file using your own EEP1 design now fully working and check that it multiplies.

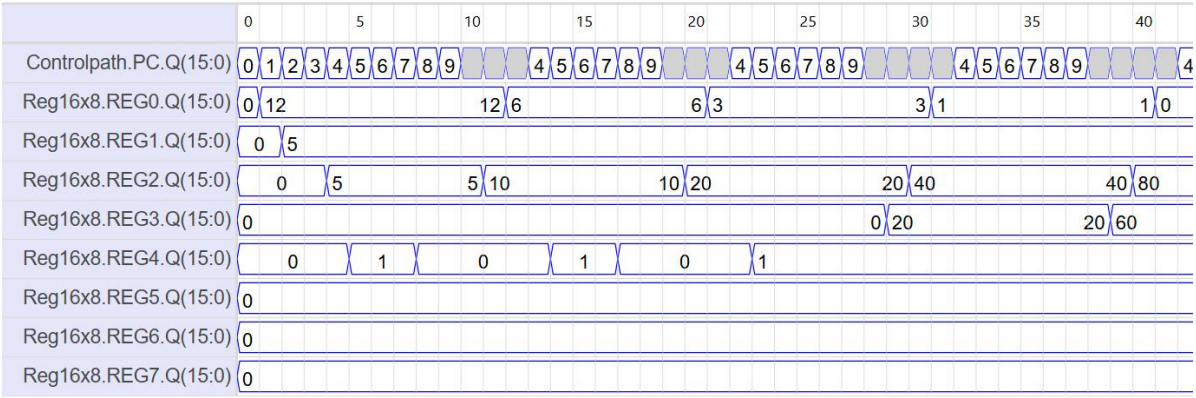
Register Allocation

- R0 = op1
- R1 = op2
- R2 = op2shifted
- R3 = sum
- R4 = tmp (temporary copy of op1 LSB)

Assembly Program (mul1.txt)

```
MOV R0, #12        ; op1 = 12
MOV R1, #5         ; op2 = 5
MOV R3, #0         ; sum = 0
MOV R2, R1         ; op2shifted = op2
MOV R4, #1         ; tmp = 1 (mask for LSB test)
; --- while (op1 != 0) ---
CMP R0, #0
JEQ 7              ; if op1 == 0, exit loop
; --- if (op1 & 1) ---
AND R4, R0, R4     ; tmp = op1 & 1  (isolate LSB)
CMP R4, #0
JEQ 2              ; if LSB == 0, skip add
ADD R3, R3, R2     ; sum = sum + op2shifted
; --- end if ---
LSL R2, R2, #1     ; op2shifted <<= 1
LSR R0, R0, #1     ; op1 >>= 1
MOV R4, #1         ; restore mask
JMP -9             ; back to while test
; --- end while ---
```

Simulation Result

Wave simulation with op1 = 12, op2 = 5 confirms R3 = 60 after the loop terminates (op1 reaches 0).

The waveform shows R2 (op2shifted) doubling each iteration (5 → 10 → 20 → 40 → 80), R0 (op1) halving (12 → 6 → 3 → 1 → 0), and R3 (sum) accumulating 0 → 20 → 60 at rounds 3 and 4.